

INTERLOCK: Least-Privilege Tool Surfaces for LLM Agent Configurations

ANONYMOUS AUTHOR(S)

Additional Key Words and Phrases: LLM agents, Model Context Protocol, tool surface, least privilege, configuration analysis, information flow, prompt injection, empirical study

ACM Reference Format:

Anonymous Author(s). 2026. INTERLOCK: Least-Privilege Tool Surfaces for LLM Agent Configurations. 1, 1 (September 2026), 5 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

An LLM agent obtains its tools from providers that a user installs. A configuration file names a set of Model Context Protocol (MCP) servers, the command or endpoint that starts each one, the credentials it receives and, for some servers, flags that decide which of its tools are advertised; skill bundles may register further servers of their own [6, 14]. The configuration is therefore the unit of deployment, and it decides the agent’s *tool surface*: the set of tools the model can see and call in a session. Two properties of that arrangement matter for security. Everything on the surface is reachable, because the agent selects tools by reading advertised names, descriptions and schemas, and an attacker who controls any content the agent reads can steer that selection [29]; a tool the user never intends to use is as callable as one they rely on. And the surface is chosen by the user rather than the provider, so the user is the party who can narrow it – yet the only narrowing action hosts make obvious is removing a server altogether.

Four strands of research protect this layer. The first inspects a single tool: hidden instructions, shadowed names and exfiltration endpoints are detected in advertised descriptions [9], trusted descriptions are regenerated to prevent poisoning [27], and benchmarks over real-world servers quantify how often such attacks succeed [12, 23, 26]. The second inspects a single provider: taint-style vulnerabilities inside MCP servers are found by static analysis [21], tool behaviour is audited at run time [22], and a sound capability bound over a resource lattice is computed by abstract interpretation for agent skills [2]. The third judges a *set* of providers: the lethal-trifecta framing names private data, untrusted content and external communication as the dangerous combination [24], toxic-flow analysis instantiates and scores tool sequences that would violate a policy at run time [1], compositional policies revoke a session when accumulated exposure would make the next call a violation [18], and reachability over a skill-induced transition system is paired with a capability-restricting cut [25]; attack work has moved to the same object, dispersing payloads across skills to evade per-artifact scanners [11, 28] and assembling legitimate tools into coordinated workflows [29]. The fourth constrains the agent at run time: control and data flow are extracted from the trusted query so that untrusted data cannot affect program flow [5], confidentiality and integrity labels are tracked and enforced at each call [4], per-call privilege policies enforce least privilege [19], plans are checked across queries [7], and zero-trust guardrails wrap the agent

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

loop [16]. Large-scale measurement completes the picture, reporting vulnerability prevalence across marketplace artifacts [8, 14], credential leakage by skills [3], and supply-chain poisoning of skill ecosystems [13, 17].

These advances share an operating point: they judge or constrain the tool surface *as the provider ships it*. Per-tool and per-provider analyses ask whether an artifact is malicious or vulnerable. Set-level analyses ask whether the installed combination is dangerous, and answer with an alert or, at run time, with a restriction that lasts for a session. Run-time enforcement asks whether the current call is permitted. What none of them computes is the question the user is actually able to act on: *which part of the surface they installed can be switched off, in the configuration itself, without losing the work they installed it for*. This question is not a repackaging of the others. Its input is a concrete configuration rather than an artifact, its output is a configuration rather than a verdict, its constraint is the user's own tasks, and its enforcement needs no change to the host, because hosts already accept per-tool allow and deny lists and servers already accept mode flags.

We make the question measurable, and we measure it. Following the lethal-trifecta framing [24], we call a tool surface *trifecta-complete* when the effects reachable through its advertised tools include a private-data source, an attacker-controlled content source and an attacker-readable sink, and we call a provider *capability-bundling* when its own tools alone are trifecta-complete. Both are static properties of the surface a configuration advertises, evaluated under an agent that may pass any tool's output into any tool's input because injected content steers its choices; neither property asserts that a deployment has been attacked or that an attack would succeed. Our preliminary study collects 19,777 deduplicated public configuration files from GitHub and derives the effect of every tool of the 17 most frequently configured servers from the servers' own source code, recording a file-and-line citation for each effect and applying the mode flags and arguments each configuration actually sets. Of the 880 configurations that consist entirely of those servers — one in ten of the multi-server configurations we collected, the restriction being that we summarise only servers whose code we read — **84.1%** (740) are trifecta-complete, and that figure moves by less than two points when the effects we could not settle from code are pushed to either extreme. In *every* one of the 740, a single server is already capability-bundling, so cross-provider composition is never the operative factor; the share of configurations that need composition to become trifecta-complete stays at or below 3.3% when we withdraw the private-data effect that browser and fetch servers derive from reaching local and intranet URLs. The drivers are ordinary, popular servers: a browser-automation server whose default surface includes a documented remote-code-execution tool, a code-hosting server that reads private repositories, reads third-party issue text and writes public comments, and a component-registry server that fetches agent-chosen URLs and reads local files. Mitigations exist but are unused or absent: read-only mode appears in 3 of 189 entries for the code-hosting server, and no flag disables the browser server's execution tool. The surface is also not static — 91.3% of stdio entries launch without a version pin, and published servers change their tool lists frequently [10] — so any narrowing must be re-checkable rather than one-off.

Neither the property nor the single-provider case is ours to claim first, and one prior number deserves a direct comparison. A registry-scale census of 12,230 tools across 1,360 servers reports that servers can hold all three capabilities at once and then complete an attack without cross-server coordination, but counts only two such servers [29]; a shipping configuration scanner already separates the single-server case from the cross-server one as distinct findings, without publishing how often either occurs [15]; and a vendor scanner states that most agents possess all three characteristics and therefore analyses each component separately [20]. Our measurement differs from the census in population and in what an effect means. The census samples servers uniformly from registries, where most entries are niche, whereas a configuration corpus weights servers by how often people install them: 9 of the 17 most frequently configured servers are

capability-bundling, including the browser, code-hosting, database and component-registry servers that appear in thousands of configurations. And because we derive effects from server source code rather than from tool metadata, we count effects a metadata-level classifier does not see, such as host code execution reachable from an advertised tool, and page or file content reachable because a browser or fetch tool accepts local and intranet URLs. The two results are therefore consistent in direction — a single server can be enough — and differ by an order of magnitude in rate because popularity and code-level effects both push the rate up. What follows for a defence is the same either way: severing cross-provider flows would leave 84.1% of these configurations trifecta-complete, so the remedy has to act inside a provider’s advertised surface.

The natural remedies fail for reasons the same measurement makes concrete. Removing a server is the only obvious control and is far too coarse: the servers that drive the verdict are the ones users installed on purpose. Server-provided modes are coarse and often missing, and when present they are rarely used. Alerting is uninformative at this base rate: a set-level trifecta check flags 84% of these configurations, which is a warning a user can only dismiss. Run-time information-flow control and privilege enforcement are strong but answer a different question: they require replacing or instrumenting the agent loop, they decide per query, and they leave the installed surface untouched, so a user of a stock host cannot apply them and an attacker who reaches the model still faces the full surface. Finally, minimising tools for accuracy or context length, as agent frameworks do, optimises an unrelated objective and offers no guarantee about effects. What is needed is a decision procedure over the configuration itself, with the user’s tasks as the constraint.

We present INTERLOCK, which computes a least-privilege tool surface for a given agent configuration in three stages. *Summarize* derives a per-tool effect summary from evidence ranked by trust: the server’s source code and the configuration’s own arguments first, declared schemas and annotations next, and free-text descriptions never used to lower an effect, only to raise it, because descriptions are attacker-controlled. *Decide* evaluates the deployer’s flow policy over the surface induced by the configuration and returns the witnessing tools for each forbidden flow, distinguishing the flows a single provider completes from those that need several. *Repair* synthesises a minimum-cost patch over the controls that hosts and servers actually expose — per-tool deny entries, server mode and toolset flags, argument narrowing, and, as the most expensive option, server removal — subject to preserving the tasks the deployment declares, and emits it as a configuration diff the user can apply. Because the surface changes when a provider updates, INTERLOCK re-decides from cached summaries when the configuration or an advertised surface changes, and reports whether the previous patch still holds.

We will evaluate INTERLOCK on the configurations and servers of our measurement, against the controls users have today and against the strongest published alternatives. We ask whether the synthesised patch removes the forbidden flows that a dynamic harness can otherwise exploit, at what cost in declared-task success, how that trade-off compares with whole-server removal, with server read-only modes, and with run-time privilege enforcement under the same policy, and how much of the patch survives provider updates. We report [A]% of trifecta-complete configurations repaired with [B]% of declared tasks preserved, against [C]% task preservation for the removal baseline.

We claim three contributions. *Observation*: prior work names the dangerous combination and observes that one server can hold it, but reports it over registries rather than deployments; we give the first measurement over real configurations, defining trifecta-completeness and capability bundling operationally and evaluating them on 19,777 public configurations with code-derived, evidence-carrying effect summaries for the 17 most-configured servers, which shows that 84.1% of the analysable configurations are trifecta-complete, that a single provider suffices in every such case, and that the mitigations those providers offer are missing or unused. *New technique*: we formulate

least-privilege tool-surface synthesis for a configuration — a minimum-cost patch over host and server controls subject to task preservation — and present INTERLOCK, whose technical contributions are trust-ranked effect summarisation that never trusts attacker-writable prose, witness-guided repair that re-decides after each candidate patch, and re-decision against cached summaries when a provider changes its surface. *Evaluation*: we measure repair effectiveness and task preservation against removal, server modes and run-time enforcement, on real configurations executed with real servers, and we release the tool, the configuration corpus and the adjudicated effect summaries.

References

- [1] Luca Beurer-Kellner, Marco Milanta, and Marc Fischer. 2025. Invariant Labs exposes novel prompt injection attack vulnerabilities, “toxic flows”, in agentic systems and MCP servers. <https://invariantlabs.ai/blog/toxic-flow-analysis>. Accessed 2026-09-16.
- [2] Varun Pratap Bhardwaj. 2026. Formal analysis and supply chain security for agentic AI skills. *arXiv preprint arXiv:2603.00195* (2026).
- [3] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. 2026. How your credentials are leaked by LLM agent skills: An empirical study. *arXiv preprint arXiv:2604.03070* (2026).
- [4] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643* (2025).
- [5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813* (2025).
- [6] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. 2026. Systematization of knowledge: Security and safety in the Model Context Protocol ecosystem. In *Intelligent Systems Conference*. Springer, 494–517.
- [7] Dylan Girrens and Guangjing Wang. 2026. SPA: Securing Persistent LLM Agents Across Queries with Plan-First Information-Flow Control. *arXiv preprint arXiv:2608.27234* (2026).
- [8] Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. 2026. Context Matters: Repository-Aware Security Analysis of the Agent Skill Ecosystem. *arXiv preprint arXiv:2603.16572* (2026).
- [9] Gunjan Jain. 2026. *Detecting MCP Tool Poisoning and Rug-Pull Attacks in LLM Agent Architectures*. Technical Report. QCrypton.
- [10] Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. 2026. MCP-Evol-Bench: Benchmarking LLM agent performance across dynamic evolutions of MCP servers. *arXiv preprint arXiv:2607.14642* (2026).
- [11] Liwei Liu, Tianzhu Han, Zijian Liu, Zishu Dong, and Na Ruan. 2026. ShareLock: A Stealthy Multi-Tool Threshold Poisoning Attack Against MCP. *arXiv preprint arXiv:2606.27027* (2026).
- [12] Shi Liu, Xuehai Tang, Xikang Yang, Liang Lin, Biyu Zhou, Wenjie Xiao, and Wantao Liu. 2026. When the Manual Lies: A Realistic Benchmark to Evaluate MCP Poisoning Attacks for LLM Agents. In *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2090–2095.
- [13] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. 2026. "Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild. *arXiv preprint arXiv:2602.06547* (2026).
- [14] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338* (2026).
- [15] MCPAudit contributors. 2026. MCPAudit: Security auditing for MCP server configurations. <https://github.com/saagpatel/MCPAudit>. Accessed 2026-09-16; issue codes MCP013 (single-server trifecta) and MCP014 (cross-server trifecta).
- [16] Nandhakumar Murugan. 2026. AgentShield: A Zero-Trust Runtime Guardrail Architecture for Autonomous Multi-Agent AI Systems with Bidirectional Context Synchronization. In *Zenodo (CERN European Organization for Nuclear Research)*. doi:10.5281/zenodo.22259021
- [17] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-chain poisoning attacks against LLM coding agent skill ecosystems. *arXiv preprint arXiv:2604.03081* (2026).
- [18] Chris Schneider, Kriti Faujdar, Philipp Schoenegger, and Ben Bariach. 2026. Securing multi-tool AI agent chains with dynamic, real-time compositional policies. *arXiv preprint arXiv:2607.03423* (2026).

- [19] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703* (2025).
- [20] Snyk. 2026. agent-scan issue codes. <https://github.com/snyk/agent-scan/blob/main/docs/issue-codes.md>. Accessed 2026-09-16.
- [21] Pengyu Sun, Zifeng Kang, Qishu Jin, Enhao Huang, Xin Liu, Dakun Shen, and Song Li. 2026. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers. *arXiv preprint arXiv:2605.21392* (2026).
- [22] Zhuoran Tan, Run Hao, Jeremy Singer, Yutian Tang, and Christos Anagnostopoulos. 2026. SandScope: a Behavioral Audit Layer for MCP Tools in LLM Agent Supply Chains. (2026).
- [23] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2026. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 35811–35819.
- [24] Simon Willison. 2025. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>. Accessed 2026-09-16.
- [25] Wujie Xiong, Rabimba Karanjai, Yang Lu, Weidong Shi, and Lei Xu. 2026. Reachability-Based Capability Confinement for LLM Agents under Indirect Prompt Injection. *arXiv preprint arXiv:2608.30041* (2026).
- [26] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. 2025. MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols. *arXiv preprint arXiv:2508.13220* (2025).
- [27] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. 2026. Trustdesc: Preventing tool poisoning in llm applications via trusted description generation. *arXiv preprint arXiv:2604.07536* (2026).
- [28] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColluSkill: Adversarial cross-skill composition for evading agent skill scanners. *arXiv preprint arXiv:2608.09732* (2026).
- [29] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *2026 IEEE Symposium on Security and Privacy (SP)*.